DataLogger de Contaminación Ambiental

Explicación general y análisis bloque por bloque
Proyecto 2 - INFO1128, Unidad II

Python • Lazarus / Pascal • Comunicación serial • Flask • SQLite • Visualización

Documento técnico orientado a estudio y exposición
Incluye arquitectura, flujo de datos, componentes Lazarus, código Python y explicación de las líneas importantes.

1. Resumen general del proyecto

El sistema recibe mediciones de cinco Unidades de Monitoreo Ambiental (UMA) simuladas por un sensor serial. El simulador genera tramas CSV por la puerta COM1. VSPE entrega esos bytes por el par virtual COM1–COM2. Lazarus lee COM2, interpreta cada trama, muestra gráficas y, si está activada la opción, envía los datos a Flask mediante HTTP POST. Flask guarda los registros en SQLite y los presenta mediante API JSON, gráficas PNG y un dashboard web.

Arquitectura del flujo de datos

simu_trama.py → COM1 → VSPE → COM2 → unit1.pas → SQLite mediante Flask → dashboard

Línea / Bloque	Explicación
1. Generación	El simulador crea valores aleatorios para material particulado, histograma, temperatura y humedad.
2. Transporte serial	VSPE conecta COM1 y COM2. La trama termina con salto de línea para permitir delimitación.
3. Recepción Lazarus	TLazSerial ejecuta MySerialRxData cada vez que llegan bytes.
4. Interpretación	ProcesarTrama separa los valores por comas y los convierte a enteros.
5. Almacenamiento	Send_Data_To_Flask convierte los datos a JSON y los envía mediante POST.
6. Presentación	Flask guarda en datalogger.db y sirve API REST, gráficos y dashboard.

2. Trama serial y significado de los campos

`id,mp01,mp25,mp10,h03,h05,h10,h25,h50,h100,te,hr`

Línea / Bloque	Explicación
id	Identificador de la UMA: 1, 2, 3, 4 o 5.
mp01, mp25, mp10	Concentraciones de partículas PM1.0, PM2.5 y PM10, expresadas en µg/m³.
h03, h05, h10, h25, h50, h100	Conteos de partículas por diámetros 0.3, 0.5, 1.0, 2.5, 5.0 y 10 µm.
te	Temperatura ambiental en grados Celsius.
hr	Humedad relativa en porcentaje.

3. simulador: simu_trama.py

Este programa crea datos simulados, los transforma a una línea de texto y los escribe en el puerto COM1. La lista de UMAs se recorre de 1 a 5.

```
0001  import argparse
0002  import random as ra
0003  import sys
0004  import time as ti
0005  import serial as rs
0006
0007
0008  def make_data(uma_id):
0009      mp01 = ra.randint(5, 100)
0010      mp25 = ra.randint(5, 100)
0011      mp10 = ra.randint(5, 100)
0012      h03 = ra.randint(10, 10000)
0013      h05 = ra.randint(10, 10000)
0014      h10 = ra.randint(10, 10000)
0015      h25 = ra.randint(10, 10000)
0016      h50 = ra.randint(10, 10000)
0017      h100 = ra.randint(10, 10000)
0018      te = ra.randint(5, 35)
0019      hr = ra.randint(20, 80)
0020
0021      return f"{uma_id},{mp01},{mp25},{mp10},{h03},{h05},{h10},{h25},{h50},{h100},{te},{hr}\n"
0022
0023
0024  def main():
0025      parser = argparse.ArgumentParser(description='Simulador de trama UMA serial')
0026      parser.add_argument('puerto', nargs='?', default='COM1', help='Puerto serial (default: COM1)')
0027      parser.add_argument('intervalo', nargs='?', default=0.1, type=float, help='Intervalo en
segundos (default: 0.1)')
0028      args = parser.parse_args()
0029
0030      print(f"[SIMULADOR] Puerto: {args.puerto} | Intervalo: {args.intervalo}s | 9600-8N1")
0031      print("[SIMULADOR] Presiona Ctrl+C para detener")
```

```
0032
0033      try:
0034          cnx = rs.Serial(args.puerto, baudrate=9600, timeout=1)
0035      except Exception as e:
0036          print(f"[ERROR] No se pudo abrir {args.puerto}: {e}")
0037          sys.exit(1)
0038
0039      uma_id = 1
0040      total = 0
0041
0042      try:
0043          while True:
0044              trama = make_data(uma_id)
0045              print(f"[{total:06d}] {trama}", end='')
0046              cnx.write(trama.encode('utf-8'))
0047              ti.sleep(args.intervalo)
0048              uma_id = 1 if uma_id >= 5 else uma_id + 1
0049              total += 1
0050      except KeyboardInterrupt:
0051          print(f"\n[SIMULADOR] Detenido. Total enviadas: {total}")
0052      finally:
0053          cnx.close()
0054
0055
0056  if __name__ == '__main__':
0057      main()
```

Explicación línea por línea de los bloques principales

Línea / Bloque	Explicación
1–5	Importan bibliotecas: parseo de argumentos, números aleatorios, salida del proceso, espera y acceso serial.
8–22, make_data()	Genera 11 valores aleatorios. La cadena f utiliza coma como separador y añade \n como terminador.
25–28, main()	Configura ArgumentParser. Permite puerto e intervalo opcionales; COM1 y 0.1 segundos son valores por defecto.
31–32	Informa por consola la configuración efectiva.
35–39	Abre el puerto virtual. baudrate=9600, timeout=1. Ante error imprime el problema y finaliza.
41–42	uma_id inicia en 1 y el contador total inicia en 0.
45–52	Ciclo infinito: crea trama, la muestra, la codifica en UTF-8, la envía y espera el intervalo.
53–54	Al llegar a 5, una trama reinicia el identificador a 1; si no, lo incrementa.
56–57, KeyboardInterrupt	Ctrl+C corta el ciclo y muestra cuántas tramas fueron enviadas.
58–59, finally	Cierra el puerto siempre, incluso después de una interrupción.

4. Punto de entrada Lazarus

```
0001  program Lazarus_client;
0002
0003  {$mode objfpc}{$H+}
0004
0005  uses
0006    {$IFDEF UNIX}
0007    cthreads,
0008    {$ENDIF}
0009    {$IFDEF HASAMIGA}
0010    athreads,
0011    {$ENDIF}
0012    Interfaces, // this includes the LCL widgetset
0013    Forms, Unit1, LazSerialPort
0014    { you can add units after this };
0015
0016  {$R *.res}
0017
0018  begin
0019    RequireDerivedFormResource:=True;
0020    Application.Scaled:=True;
0021    {$PUSH}{$WARN 5044 OFF}
0022    Application.MainFormOnTaskbar:=True;
0023    {$POP}
0024    Application.Initialize;
0025    Application.CreateForm(TForm1, Form1);
0026    Application.Run;
0027  end.
0028
```

Lazarus_client.lpr es el programa principal. **Interfaces** carga el widget gráfico de Windows, **Forms** administra ventanas y Unit1 contiene el formulario y su lógica. Application.CreateForm crea el formulario TForm1, y Application.Run inicia el ciclo de eventos.

5. Formularios Lazarus: unit1.pas

Este archivo contiene la lógica principal: configuración de componentes, recepción serial, filtrado por UMA, actualización de gráficos y envío HTTP.

```
0001  unit Unit1;
0002
0003  {$mode objfpc}{$H+}
0004
0005  interface
0006
0007  uses
0008    Classes, SysUtils, Forms, Controls, Graphics, Dialogs,
0009    StdCtrls, ExtCtrls, ComCtrls, Buttons, Spin,
0010    TAGraph, TASeries, TAChartAxisUtils, TACustomSeries,
0011    LazSerial, fphttpclient, fpjson, jsonparser,
0012    Grids, Math;
0013
0014  const
0015    MAX_SAMPLES = 12;
0016    MAX_HIST    = 6;
0017
0018  type
0019    TdataArray = array[0..MAX_SAMPLES-1] of Integer;
0020    THistArray = array[0..MAX_HIST-1] of Integer;
0021
0022    { TForm1 }
0023    TForm1 = class(TForm)
0024      // -- Serial
0025      MySerial: TLazSerial;
0026      // -- Timer UI
0027      tmrUI: TTimer;
0028      // -- Main chart (line series)
0029      ChartPM: TChart;
0030      SerieMP01: TLineSeries;
0031      SerieMP25: TLineSeries;
0032      SerieMP10: TLineSeries;
0033      // -- Histogram chart
0034      ChartHist: TChart;
0035      HistBar: TBarSeries;
0036      // -- Panel top
0037      pnlTop: TPanel;
0038      lblTitle: TLabel;
0039      // -- Panel COM
0040      pnlCom: TPanel;
0041      lblCOM: TLabel;
0042      edtCOM: TEdit;
0043      btnConnect: TBitBtn;
0044      // -- Panel UMA
0045      pnlUMA: TPanel;
0046      lblUMA: TLabel;
0047      cmbUMA: TComboBox;
0048      // -- Panel Flask
0049      pnlFlask: TPanel;
0050      lblFlask: TLabel;
0051      edtFlask: TEdit;
0052      chkSendFlask: TCheckBox;
0053      // -- Panel T/HR
0054      pnlTHR: TPanel;
0055      lblTE: TLabel;
0056      lblHR: TLabel;
0057      lblTEVal: TLabel;
0058      lblHRVal: TLabel;
0059      // -- Status
0060      pnlStatus: TPanel;
0061      lblStatus: TLabel;
0062      // -- Memo log
0063      memLog: TMemo;
0064
0065      procedure FormCreate(Sender: TObject);
0066      procedure FormDestroy(Sender: TObject);
0067      procedure btnConnectClick(Sender: TObject);
0068      procedure MySerialRxData(Sender: TObject);
0069      procedure cmbUMAChange(Sender: TObject);
0070      procedure tmrUITimer(Sender: TObject);
0071
0072    private
0073      // Data arrays
0074      aMP01 : TdataArray;
0075      aMP25 : TdataArray;
0076      aMP10 : TdataArray;
0077      aHisto: THistArray;
0078      aClr  : array[0..MAX_HIST-1] of TColor;
0079      // Seleccion UMA
0080      nUMA_Sel : Integer;
0081      sBufSerial: String;
0082      bConectado: Boolean;
0083      bNuevosDatos: Boolean;
0084
```

```
0085      procedure InicializarArreglos;
0086      procedure ProcesarTrama(const sTrama: String);
0087      procedure ActualizarGrafica;
0088      procedure ActualizarHistograma;
0089      procedure LimpiarSeries;
0090      procedure Send_Data_To_Flask(nId: Integer; mp01,mp25,mp10,
0091                                   h03,h05,h10,h25,h50,h100,te,hr: Integer);
0092      procedure Log(const sMsg: String);
0093    public
0094    end;
0095
0096  var
0097    Form1: TForm1;
0098
0099  implementation
0100
0101  {$R *.lfm}
0102
0103  { TForm1 }
0104
0105  procedure TForm1.FormCreate(Sender: TObject);
0106  var
0107    i: Integer;
0108  begin
0109    Caption := 'DataLogger - Contaminación Ambiental (UMA1-UMA5)';
0110    bConectado   := False;
0111    bNuevosDatos := False;
0112    sBufSerial   := '';
0113    nUMA_Sel     := 1;
0114
0115    aClr[0] := clRed;
0116    aClr[1] := clFuchsia;
0117    aClr[2] := clBlue;
0118    aClr[3] := clGreen;
0119    aClr[4] := clYellow;
0120    aClr[5] := TColor($000099FF);
0121
0122    InicializarArreglos;
0123
0124    cmbUMA.Items.Clear;
0125    for i := 1 to 5 do
0126      cmbUMA.Items.Add('UMA' + IntToStr(i));
0127    cmbUMA.ItemIndex := 0;
0128
0129    edtCOM.Text  := 'COM2';
0130    edtFlask.Text := 'http://127.0.0.1:5000/datos';
0131
0132    lblStatus.Caption := 'Desconectado';
0133  end;
0134
0135  procedure TForm1.FormDestroy(Sender: TObject);
0136  begin
0137    if bConectado then
0138    begin
0139      MySerial.Active := False;
0140      bConectado := False;
0141    end;
0142  end;
0143
0144  procedure TForm1.InicializarArreglos;
0145  var
0146    i: Integer;
0147  begin
0148    for i := 0 to MAX_SAMPLES-1 do
0149    begin
0150      aMP01[i] := 0;
0151      aMP25[i] := 0;
0152      aMP10[i] := 0;
0153    end;
0154    for i := 0 to MAX_HIST-1 do
0155      aHisto[i] := 0;
0156  end;
0157
0158  procedure TForm1.Log(const sMsg: String);
0159  begin
0160    if memLog.Lines.Count > 200 then
0161      memLog.Lines.Delete(0);
0162    memLog.Lines.Add(FormatDateTime('hh:nn:ss', Now) + ' ' + sMsg);
0163  end;
0164
0165  procedure TForm1.btnConnectClick(Sender: TObject);
0166  begin
0167    if not bConectado then
0168    begin
0169      try
0170        MySerial.Device   := edtCOM.Text;
0171        MySerial.BaudRate := br__9600;
0172        MySerial.DataBits := db8bits;
0173        MySerial.Parity   := pNone;
0174        MySerial.StopBits := sbOne;
0175        MySerial.FlowControl := fcNone;
```

```
0176        MySerial.Active   := True;
0177        bConectado := True;
0178        btnConnect.Caption := 'Desconectar';
0179        lblStatus.Caption  := 'Conectado a ' + edtCOM.Text;
0180        lblStatus.Font.Color := clGreen;
0181        Log('Conectado a ' + edtCOM.Text);
0182      except
0183        on E: Exception do
0184        begin
0185          ShowMessage('Error al conectar: ' + E.Message);
0186          Log('ERROR al conectar: ' + E.Message);
0187        end;
0188      end;
0189    end
0190    else
0191    begin
0192      MySerial.Active := False;
0193      bConectado := False;
0194      btnConnect.Caption := 'Conectar';
0195      lblStatus.Caption  := 'Desconectado';
0196      lblStatus.Font.Color := clRed;
0197      Log('Desconectado');
0198    end;
0199  end;
0200
0201  procedure TForm1.MySerialRxData(Sender: TObject);
0202  var
0203    sRecv: String;
0204    p: Integer;
0205  begin
0206    sRecv := MySerial.ReadData;
0207    sBufSerial := sBufSerial + sRecv;
0208
0209    repeat
0210      p := Pos(#10, sBufSerial);
0211      if p > 0 then
0212      begin
0213        ProcesarTrama(Trim(Copy(sBufSerial, 1, p - 1)));
0214        Delete(sBufSerial, 1, p);
0215      end;
0216    until p = 0;
0217  end;
0218
0219  procedure TForm1.ProcesarTrama(const sTrama: String);
0220  var
0221    aTokens: TStringList;
0222    nId, mp01, mp25, mp10: Integer;
0223    h03, h05, h10, h25, h50, h100, te, hr: Integer;
0224    i: Integer;
0225  begin
0226    if sTrama = '' then Exit;
0227
0228    aTokens := TStringList.Create;
0229    try
0230      aTokens.Delimiter := ',';
0231      aTokens.StrictDelimiter := True;
0232      aTokens.DelimitedText := sTrama;
0233
0234      if aTokens.Count < 12 then Exit;
0235
0236      nId  := StrToIntDef(aTokens[0],  0);
0237      mp01 := StrToIntDef(aTokens[1],  0);
0238      mp25 := StrToIntDef(aTokens[2],  0);
0239      mp10 := StrToIntDef(aTokens[3],  0);
0240      h03  := StrToIntDef(aTokens[4],  0);
0241      h05  := StrToIntDef(aTokens[5],  0);
0242      h10  := StrToIntDef(aTokens[6],  0);
0243      h25  := StrToIntDef(aTokens[7],  0);
0244      h50  := StrToIntDef(aTokens[8],  0);
0245      h100 := StrToIntDef(aTokens[9],  0);
0246      te   := StrToIntDef(aTokens[10], 0);
0247      hr   := StrToIntDef(aTokens[11], 0);
0248
0249      if chkSendFlask.Checked then
0250        Send_Data_To_Flask(nId, mp01, mp25, mp10, h03, h05, h10, h25, h50, h100, te, hr);
0251
0252      if nId <> nUMA_Sel then Exit;
0253
0254      for i := 0 to MAX_SAMPLES-2 do
0255      begin
0256        aMP01[i] := aMP01[i+1];
0257        aMP25[i] := aMP25[i+1];
0258        aMP10[i] := aMP10[i+1];
0259      end;
0260      aMP01[MAX_SAMPLES-1] := mp01;
0261      aMP25[MAX_SAMPLES-1] := mp25;
0262      aMP10[MAX_SAMPLES-1] := mp10;
0263
0264      aHisto[0] := h03;
0265      aHisto[1] := h05;
0266      aHisto[2] := h10;
```

```
0267      aHisto[3] := h25;
0268      aHisto[4] := h50;
0269      aHisto[5] := h100;
0270
0271      lblTEVal.Caption := IntToStr(te) + ' °C';
0272      lblHRVal.Caption := IntToStr(hr) + ' %';
0273
0274      bNuevosDatos := True;
0275      Log(Format('UMA%d | PM1:%d PM2.5:%d PM10:%d | T:%d H:%d', [nId,mp01,mp25,mp10,te,hr]));
0276
0277
0278    finally
0279      aTokens.Free;
0280    end;
0281  end;
0282
0283  procedure TForm1.LimpiarSeries;
0284  begin
0285    SerieMP01.Clear;
0286    SerieMP25.Clear;
0287    SerieMP10.Clear;
0288  end;
0289
0290  procedure TForm1.ActualizarGrafica;
0291  var
0292    i: Integer;
0293  begin
0294    LimpiarSeries;
0295    for i := 0 to MAX_SAMPLES-1 do
0296    begin
0297      SerieMP01.AddXY(i, aMP01[i]);
0298      SerieMP25.AddXY(i, aMP25[i]);
0299      SerieMP10.AddXY(i, aMP10[i]);
0300    end;
0301  end;
0302
0303  procedure TForm1.ActualizarHistograma;
0304  var
0305    i: Integer;
0306    aLabels: array[0..5] of String = ('0.3','0.5','1.0','2.5','5.0','10');
0307  begin
0308    HistBar.Clear;
0309    for i := 0 to MAX_HIST-1 do
0310      HistBar.Add(aHisto[i], aLabels[i], aClr[i]);
0311  end;
0312
0313  procedure TForm1.tmrUITimer(Sender: TObject);
0314  begin
0315    if bNuevosDatos then
0316    begin
0317      ActualizarGrafica;
0318      ActualizarHistograma;
0319      bNuevosDatos := False;
0320    end;
0321  end;
0322
0323  procedure TForm1.cmbUMAChange(Sender: TObject);
0324  begin
0325    nUMA_Sel := cmbUMA.ItemIndex + 1;
0326    InicializarArreglos;
0327    LimpiarSeries;
0328    Log('Cambiado a UMA' + IntToStr(nUMA_Sel));
0329  end;
0330
0331  procedure TForm1.Send_Data_To_Flask(nId: Integer; mp01,mp25,mp10,
0332                                      h03,h05,h10,h25,h50,h100,te,hr: Integer);
0333  var
0334    oHTTP  : TFPHTTPClient;
0335    jData  : TJSONObject;
0336    sJSON  : String;
0337    oStream: TStringStream;
0338  begin
0339    oHTTP  := TFPHTTPClient.Create(nil);
0340    jData  := TJSONObject.Create;
0341    try
0342      jData.Add('id',   nId);
0343      jData.Add('mp01', mp01);
0344      jData.Add('mp25', mp25);
0345      jData.Add('mp10', mp10);
0346      jData.Add('h03',  h03);
0347      jData.Add('h05',  h05);
0348      jData.Add('h10',  h10);
0349      jData.Add('h25',  h25);
0350      jData.Add('h50',  h50);
0351      jData.Add('h100', h100);
0352      jData.Add('te',   te);
0353      jData.Add('hr',   hr);
0354      sJSON := jData.AsJSON;
0355
0356      oHTTP.AddHeader('Content-Type','application/json');
0357      oStream := TStringStream.Create(sJSON);
```

```
0358      try
0359        oHTTP.RequestBody := oStream;
0360        oHTTP.Post(edtFlask.Text);
0361        Log('Flask OK: enviado ' + IntToStr(Length(sJSON)) + ' bytes');
0362      finally
0363        oStream.Free;
0364      end;
0365    except
0366      on E: Exception do
0367        Log('Flask ERROR: ' + E.Message);
0368    end;
0369    jData.Free;
0370    oHTTP.Free;
0371  end;
0372
0373  end.
```

Explicación bloque por bloque

Línea / Bloque	Explicación
1–4, unit y mode	Declara Unit1 y usa Object Pascal para Free Pascal/Lazarus. {$H+} habilita cadenas largas dinámicas.
6–13, uses	Carga componentes de formularios, controles, gráficos, serial, HTTP, JSON y arreglos.
15–17, constantes	MAX_SAMPLES=12 limita el historial visual. MAX_HIST=6 corresponde a los seis tamaños del histograma.
19–21, arreglos	TdataArray guarda series PM; THistArray guarda los seis conteos.
24–63, TForm1	Declara el formulario y sus componentes visuales, eventos, datos privados y procedimientos.
66, VAR	Declara la instancia global Form1.
70–101, FormCreate	Inicializa estado, colores, arreglos, las cinco UMAs y los valores predeterminados COM2 y Flask.
103–112, FormDestroy	Cierra el puerto serial si el formulario se destruye mientras está conectado.
114–139, arreglos y Log	Inicializa las muestras y conserva solo los últimos 200 mensajes del registro.
141–179, conectar	Abre/cierra COM2 con 9600 baudios, 8 bits, sin paridad, un bit de parada y sin control de flujo.
181–198, MySerialRxData	Lee los bytes disponibles, los agrega al buffer y procesa cada segmento terminado por LF.
200–261, ProcesarTrama	Separa por coma, convierte doce campos y envía a Flask antes de filtrar por UMA seleccionada.
263–281, gráficos	Limpia y vuelve a dibujar las 12 muestras de PM1.0, PM2.5 y PM10.
283–294, histograma	Convierte los seis conteos a barras etiquetadas por diámetro.
296–305, tmrUITimer	El Timer traslada el procesamiento visual fuera de MySerialRxData; se ejecuta cada 200 ms.
307–315, cmbUMAChange	Actualiza nUMA_Sel, reinicia los arreglos y registra el cambio.
317–357, Send_Data_To_Flask	Construye un objeto JSON con todos los campos, establece Content-Type y hace POST al URL configurado.

Línea crítica: recepción y ensamblado serial

Línea / Bloque	Explicación
sRecv := MySerial.ReadData	Extrae los bytes que llegaron al puerto.
sBufSerial := sBufSerial + sRecv	Une el fragmento recibido con bytes anteriores, porque una trama puede llegar dividida.
Pos(#10, sBufSerial)	Busca el salto de línea LF que marca el final de una trama.
ProcesarTrama(...)	Envía al parser solamente el texto completo antes de LF.
Delete(sBufSerial, 1, p)	Elimina de la cola la trama ya procesada y conserva cualquier contenido posterior.

Línea crítica: parseo de la trama

Línea / Bloque	Explicación
aTokens.Delimiter := ','	Define coma como separador.
StrictDelimiter := True	Impide interpretar el separador de forma flexible o incompleta.
DelimitedText := sTrama	Rellena la lista aTokens.
Count < 12	Descarta una trama incompleta para evitar leer índices inexistentes.
StrToIntDef(..., 0)	Convierte texto a entero; si el dato es inválido usa cero.

Línea / Bloque	Explicación
if nId <> nUMA_Sel then Exit	Hace visible en Lazarus solamente la UMA elegida en el ComboBox.
bNuevosDatos := True	Avisa al Timer que debe refrescar gráficos sin bloquear la recepción serial.

Línea crítica: comunicación HTTP JSON

Línea / Bloque	Explicación
TFPHTTPClient.Create(nil)	Crea el cliente HTTP incluido en Free Pascal.
TJSONObject.Create	Crea el objeto JSON que transportará la medición.
jData.Add('id', nId)	Asocia cada nombre de campo JSON con su valor entero.
jData.AsJSON	Serializa el objeto como texto JSON.
AddHeader('Content-Type', 'application/json')	Indica que el cuerpo contiene JSON.
TStringStream.Create(sJSON)	Convierte el texto JSON en el cuerpo de la petición.
oHTTP.Post(edtFlask.Text)	Realiza HTTP POST al URL configurado.
except ... on E	Captura problemas de red y los escribe en el log; evita cerrar la aplicación.

6. Diseño visual: unit1.lfm

El archivo .lfm describe los componentes en forma de propiedades Lazarus. Se conecta al formulario del código mediante los mismos nombres: MySerial, tmrUI, ChartPM, SerieMP01 y los demás controles.

```
0001  object Form1: TForm1
0002    Left = 100
0003    Height = 720
0004    Top = 50
0005    Width = 1100
0006    Caption = 'DataLogger - Contaminacion Ambiental'
0007    ClientHeight = 720
0008    ClientWidth = 1100
0009    DesignTimePPI = 96
0010    LCLVersion = '4.8.0.0'
0011    OnCreate = FormCreate
0012    OnDestroy = FormDestroy
0013    object pnlTop: TPanel
0014      Left = 0
0015      Height = 55
0016      Top = 0
0017      Width = 1100
0018      Align = alTop
0019      BevelInner = bvLowered
0020      Caption = ''
0021      Color = 1644825
0022      ParentBackground = False
0023      TabOrder = 0
0024      object lblTitle: TLabel
0025        Left = 8
0026        Height = 35
0027        Top = 10
0028        Width = 500
0029        Caption = 'DataLogger - Monitor de Contaminacion Ambiental'
0030        Font.Color = clWhite
0031        Font.Height = -21
0032        Font.Style = [fsBold]
0033        ParentFont = False
0034      end
0035      object pnlTHR: TPanel
0036        Left = 700
0037        Height = 50
0038        Top = 2
0039        Width = 390
0040        BevelOuter = bvNone
0041        Caption = ''
0042        Color = 1644825
0043        ParentBackground = False
0044        TabOrder = 0
0045        object lblTE: TLabel
0046          Left = 10
0047          Height = 14
0048          Top = 5
0049          Width = 80
0050          Caption = 'Temperatura:'
0051          Font.Color = clAqua
0052          Font.Style = [fsBold]
0053          ParentFont = False
```

```
0054        end
0055        object lblTEVal: TLabel
0056          Left = 90
0057          Height = 22
0058          Top = 2
0059          Width = 60
0060          Caption = '-- C'
0061          Font.Color = clYellow
0062          Font.Height = -16
0063          Font.Style = [fsBold]
0064          ParentFont = False
0065        end
0066        object lblHR: TLabel
0067          Left = 180
0068          Height = 14
0069          Top = 5
0070          Width = 80
0071          Caption = 'Humedad:'
0072          Font.Color = clAqua
0073          Font.Style = [fsBold]
0074          ParentFont = False
0075        end
0076        object lblHRVal: TLabel
0077          Left = 260
0078          Height = 22
0079          Top = 2
0080          Width = 60
0081          Caption = '-- %'
0082          Font.Color = clYellow
0083          Font.Height = -16
0084          Font.Style = [fsBold]
0085          ParentFont = False
0086        end
0087      end
0088    end
0089    object pnlCom: TPanel
0090      Left = 0
0091      Height = 45
0092      Top = 55
0093      Width = 1100
0094      Align = alTop
0095      BevelOuter = bvNone
0096      Caption = ''
0097      Color = clBtnFace
0098      TabOrder = 1
0099      object lblCOM: TLabel
0100        Left = 8
0101        Height = 14
0102        Top = 15
0103        Width = 60
0104        Caption = 'Puerto COM:'
0105      end
0106      object edtCOM: TEdit
0107        Left = 72
0108        Height = 23
0109        Top = 12
0110        Width = 80
0111        TabOrder = 0
0112        Text = 'COM2'
0113      end
0114      object btnConnect: TBitBtn
0115        Left = 160
0116        Height = 27
0117        Top = 10
0118        Width = 100
0119        Caption = 'Conectar'
0120        Kind = bkOK
0121        NumGlyphs = 0
0122        OnClick = btnConnectClick
0123        TabOrder = 1
0124      end
0125      object lblUMA: TLabel
0126        Left = 280
0127        Height = 14
0128        Top = 15
0129        Width = 70
0130        Caption = 'Sensor UMA:'
0131      end
0132      object cmbUMA: TComboBox
0133        Left = 355
0134        Height = 23
0135        Top = 12
0136        Width = 80
0137        ItemIndex = 0
0138        OnChange = cmbUMAChange
0139        Style = csDropDownList
0140        TabOrder = 2
0141        Items.Strings = (
0142          'UMA1'
0143          'UMA2'
0144          'UMA3'
```

```
0145          'UMA4'
0146          'UMA5'
0147        )
0148      end
0149      object chkSendFlask: TCheckBox
0150        Left = 455
0151        Height = 17
0152        Top = 14
0153        Width = 110
0154        Caption = 'Enviar a Flask'
0155        TabOrder = 3
0156      end
0157      object lblFlask: TLabel
0158        Left = 570
0159        Height = 14
0160        Top = 15
0161        Width = 80
0162        Caption = 'URL Flask:'
0163      end
0164      object edtFlask: TEdit
0165        Left = 650
0166        Height = 23
0167        Top = 12
0168        Width = 240
0169        TabOrder = 4
0170        Text = 'http://127.0.0.1:5000/datos'
0171      end
0172    end
0173    object ChartPM: TChart
0174      Left = 0
0175      Height = 320
0176      Top = 100
0177      Width = 700
0178      AxisList = <
0179        item
0180          Title.Caption = 'ug/m3'
0181          Title.LabelFont.Color = clDefault
0182          Title.Visible = True
0183        end
0184        item
0185          Alignment = calBottom
0186          Title.Caption = 'Muestras'
0187          Title.LabelFont.Color = clDefault
0188          Title.Visible = True
0189        end>
0190      BackColor = clBlack
0191      Title.Text.Strings = (
0192        'Series de Tiempo: PM1.0 / PM2.5 / PM10'
0193      )
0194      Title.Font.Color = clBlack
0195      TabOrder = 2
0196      object SerieMP01: TLineSeries
0197        AxisIndexX = 1
0198        AxisIndexY = 0
0199        ShowPoints = False
0200        LineType = ltFromPrevious
0201        Pen.Color = clRed
0202        SeriesColor = clRed
0203        Title = 'PM1.0 um'
0204      end
0205      object SerieMP25: TLineSeries
0206        AxisIndexX = 1
0207        AxisIndexY = 0
0208        ShowPoints = False
0209        LineType = ltFromPrevious
0210        Pen.Color = clGreen
0211        SeriesColor = clGreen
0212        Title = 'PM2.5 um'
0213      end
0214      object SerieMP10: TLineSeries
0215        AxisIndexX = 1
0216        AxisIndexY = 0
0217        ShowPoints = False
0218        LineType = ltFromPrevious
0219        Pen.Color = clBlue
0220        SeriesColor = clBlue
0221        Title = 'PM10 um'
0222      end
0223    end
0224    object ChartHist: TChart
0225      Left = 700
0226      Height = 320
0227      Top = 100
0228      Width = 400
0229      AxisList = <
0230        item
0231          Title.Caption = 'Conteo'
0232          Title.LabelFont.Color = clDefault
0233          Title.Visible = True
0234        end
0235        item
```

```
0236          Alignment = calBottom
0237          Title.Caption = 'Diametro (um)'
0238          Title.LabelFont.Color = clDefault
0239          Title.Visible = True
0240        end>
0241      BackColor = clBlack
0242      Title.Text.Strings = (
0243        'Histograma de Particulas'
0244      )
0245      Title.Font.Color = clBlack
0246      TabOrder = 3
0247      object HistBar: TBarSeries
0248        AxisIndexX = 1
0249        AxisIndexY = 0
0250        BarPen.Color = clDefault
0251        BarWidthPercent = 80
0252        SeriesColor = clRed
0253        Title = 'Histograma'
0254      end
0255    end
0256    object memLog: TMemo
0257      Left = 0
0258      Height = 200
0259      Top = 470
0260      Width = 1100
0261      Align = alBottom
0262      Font.Name = 'Courier New'
0263      Font.Height = -12
0264      ReadOnly = True
0265      ScrollBars = ssVertical
0266      TabOrder = 4
0267      WordWrap = False
0268    end
0269    object pnlStatus: TPanel
0270      Left = 0
0271      Height = 25
0272      Top = 695
0273      Width = 1100
0274      Align = alBottom
0275      BevelOuter = bvLowered
0276      Caption = ''
0277      TabOrder = 5
0278      object lblStatus: TLabel
0279        Left = 5
0280        Height = 14
0281        Top = 5
0282        Width = 100
0283        Caption = 'Desconectado'
0284        Font.Color = clRed
0285        Font.Style = [fsBold]
0286        ParentFont = False
0287      end
0288    end
0289    object MySerial: TLazSerial
0290      Active = False
0291      BaudRate = br__9600
0292      DataBits = db8bits
0293      Parity = pNone
0294      FlowControl = fcNone
0295      StopBits = sbOne
0296      SynSer.Tag = 0
0297      SynSer.Handle = -1
0298      SynSer.RaiseExcept = False
0299      SynSer.TestDSR = False
0300      SynSer.TestCTS = False
0301      SynSer.MaxLineLength = 0
0302      SynSer.DeadlockTimeout = 30000
0303      SynSer.LinuxLock = False
0304      SynSer.ConvertLineEnd = False
0305      SynSer.AtTimeout = 1000
0306      SynSer.InterPacketTimeout = True
0307      Device = 'COM2'
0308      RcvLineCRLF = False
0309      OnRxData = MySerialRxData
0310      Left = 20
0311      Top = 420
0312    end
0313    object tmrUI: TTimer
0314      Enabled = True
0315      Interval = 200
0316      OnTimer = tmrUITimer
0317      Left = 100
0318      Top = 420
0319    end
0320  end
```

Explicación de bloques LFM

Línea / Bloque	Explicación
object Form1	Ventana principal de 1100 × 720, nombre DataLogger.
OnCreate / OnDestroy	Vinculan los eventos con FormCreate y FormDestroy definidos en Pascal.
pnlTop	Banda superior que contiene el título, temperatura y humedad.
pnlCom	Banda de configuración: puerto, botón, UMA, casilla Flask y URL.
ChartPM	Gráfica de líneas negra; eje vertical µg/m³ y eje horizontal muestras.
SerieMP01 / MP25 / MP10	Tres series con colores rojo, verde y azul.
ChartHist	Gráfica de barras para los diámetros de partículas.
memLog	Memo inferior, tipo Courier New, lectura solamente y desplazamiento vertical.
MySerial	Componente TLazSerial: 9600-8N1, COM2 por defecto y evento OnRxData.
tmrUI	Timer activo a intervalos de 200 ms para actualizar la interfaz.

7. Servidor Flask: server_flask.py

El servidor expone una API REST, crea SQLite y transforma filas en gráficas. Los importes cargan Flask, SQLite, tiempo, imágenes y Matplotlib.

```
0001  import base64
0002  from datetime import datetime, timedelta
0003  import io
0004  import os
0005  import sqlite3
0006  import threading
0007
0008  from flask import Flask, jsonify, request, send_file
0009  import matplotlib
0010  matplotlib.use('Agg')
0011  import matplotlib.dates as mdates
0012  import matplotlib.pyplot as plt
0013
0014  app = Flask(__name__)
0015  DB_PATH = os.path.join(os.path.dirname(__file__), 'datalogger.db')
0016  plot_lock = threading.Lock()
0017
0018
0019  def init_db():
0020      con = sqlite3.connect(DB_PATH)
0021      cur = con.cursor()
0022      cur.execute('''
0023          CREATE TABLE IF NOT EXISTS mediciones (
0024              id       INTEGER PRIMARY KEY AUTOINCREMENT,
0025              uma_id   INTEGER NOT NULL,
0026              mp01     INTEGER,
0027              mp25     INTEGER,
0028              mp10     INTEGER,
0029              h03      INTEGER,
0030              h05      INTEGER,
0031              h10      INTEGER,
0032              h25      INTEGER,
0033              h50      INTEGER,
0034              h100     INTEGER,
0035              te       INTEGER,
0036              hr       INTEGER,
0037              fecha    TEXT NOT NULL
0038          )
0039      ''')
0040      con.commit()
0041      con.close()
0042
0043
0044  def get_db():
0045      con = sqlite3.connect(DB_PATH)
0046      con.row_factory = sqlite3.Row
0047      return con
0048
0049
0050  def parse_fecha(f_str):
0051      for fmt in ('%Y-%m-%d %H:%M:%S', '%Y-%m-%d %H:%M:%S.%f', '%Y-%m-%d'):
0052          try:
0053              return datetime.strptime(f_str, fmt)
0054          except Exception:
0055              pass
0056      return datetime.now()
0057
0058
0059  def generar_grafica_pm(rows, titulo='Series de Tiempo - Material Particulado'):
```

```
    with plot_lock:
        fig = plt.figure(figsize=(12, 9), facecolor='#0d1117')
        if not rows:
            ax = fig.add_subplot(111)
            ax.set_facecolor('#0d1117')
            ax.text(0.5, 0.5, f'{titulo}\n\n(Esperando datos de la estación...)',
                    ha='center', va='center', fontsize=14, color='#00d4ff')
            ax.axis('off')
        else:
            fechas = [parse_fecha(r['fecha']) for r in rows]
            mp01 = [r['mp01'] or 0 for r in rows]
            mp25 = [r['mp25'] or 0 for r in rows]
            mp10 = [r['mp10'] or 0 for r in rows]
            te = [r['te'] or 0 for r in rows]
            hr = [r['hr'] or 0 for r in rows]

            ax1 = fig.add_subplot(3, 1, 1)
            ax2 = fig.add_subplot(3, 1, 2, sharex=ax1)
            ax3 = fig.add_subplot(3, 1, 3, sharex=ax1)

            for ax in (ax1, ax2, ax3):
                ax.set_facecolor('#0d1117')
                ax.tick_params(colors='white')
                ax.spines[:].set_color('#333')
                ax.grid(True, alpha=0.25, color='#444')

            ax1.plot(fechas, mp01, color='#ff4444', linewidth=2, label='PM1.0 µg/m³', marker='o',
markersize=4)
            ax1.plot(fechas, mp25, color='#22c55e', linewidth=2, label='PM2.5 µg/m³', marker='s',
markersize=4)
            ax1.plot(fechas, mp10, color='#38bdf8', linewidth=2, label='PM10 µg/m³', marker='^',
markersize=4)
            ax1.set_ylabel('Concentración (µg/m³)', color='white')
            ax1.set_title(titulo, color='white', fontsize=13, fontweight='bold', pad=10)
            ax1.legend(loc='upper right', facecolor='#1e293b', labelcolor='white', framealpha=0.85)

            ax2.plot(fechas, te, color='#f59e0b', linewidth=2, label='Temperatura (°C)',
marker='o', markersize=4)
            ax2.set_ylabel('Temp (°C)', color='white')
            ax2.legend(loc='upper right', facecolor='#1e293b', labelcolor='white', framealpha=0.85)

            ax3.plot(fechas, hr, color='#06b6d4', linewidth=2, label='Humedad Relativa (%)',
marker='o', markersize=4)
            ax3.set_ylabel('HR (%)', color='white')
            ax3.set_xlabel('Hora de Registro', color='white')
            ax3.legend(loc='upper right', facecolor='#1e293b', labelcolor='white', framealpha=0.85)

            if len(fechas) == 1:
                t0 = fechas[0]
                ax3.set_xlim(t0 - timedelta(minutes=2), t0 + timedelta(minutes=2))

            ax3.xaxis.set_major_formatter(mdates.DateFormatter('%H:%M:%S'))
            fig.autofmt_xdate(rotation=25)

        fig.tight_layout()
        buf = io.BytesIO()
        fig.savefig(buf, format='png', facecolor=fig.get_facecolor(), dpi=100)
        plt.close(fig)
        buf.seek(0)
        return buf


def generar_histograma(uma_id=None):
    with plot_lock:
        con = get_db()
        if uma_id:
            row = con.execute('''
                SELECT AVG(h03) h03, AVG(h05) h05, AVG(h10) h10,
                       AVG(h25) h25, AVG(h50) h50, AVG(h100) h100
                FROM mediciones WHERE uma_id=?
            ''', (uma_id,)).fetchone()
            titulo = f'Histograma de Partículas — UMA{uma_id}'
        else:
            row = con.execute('''
                SELECT AVG(h03) h03, AVG(h05) h05, AVG(h10) h10,
                       AVG(h25) h25, AVG(h50) h50, AVG(h100) h100
                FROM mediciones
            ''').fetchone()
            titulo = 'Histograma de Partículas (Promedio Global)'
        con.close()

        labels = ['0.3 µm', '0.5 µm', '1.0 µm', '2.5 µm', '5.0 µm', '10 µm']
        if row and any(row[k] is not None for k in ['h03', 'h05', 'h10', 'h25', 'h50', 'h100']):
            valores = [
                row['h03'] or 0, row['h05'] or 0, row['h10'] or 0,
                row['h25'] or 0, row['h50'] or 0, row['h100'] or 0
        h2 { color: #88aaff; }
        a { color: #00ffcc; text-decoration: none; font-weight: bold; }
        a:hover { text-decoration: underline; }
        .card { background: #111827; border: 1px solid #1e3a5f; border-radius: 8px; padding: 16px;
margin: 12px 0; max-width: 700px; }
```

```
0184          code { background: #1a2a3a; padding: 2px 6px; border-radius: 4px; color: #00d4ff; }
0185          .badge { display: inline-block; background: #00d4ff; color: #000; border-radius: 4px;
padding: 2px 8px; font-weight: bold; font-size: 0.8em; }
0186      </style>
0187  </head>
0188  <body>
0189      <h1>DataLogger - Monitor de Contaminación Ambiental</h1>
0190      <p>Servidor Flask activo | Base de datos SQLite</p>
0191      <div class="card">
0192          <h2>Endpoints disponibles</h2>
0193          <ul>
0194              <li><span class="badge">POST</span> <code>/datos</code> - Registrar medición (JSON)</
li>
0195              <li><span class="badge">GET</span>  <code>/datos</code> - Listar mediciones (JSON)</li>
0196              <li><span class="badge">GET</span>  <code>/datos/uma/&lt;id&gt;</code> - Filtrar por
UMA (JSON)</li>
0197              <li><span class="badge">GET</span>  <code>/datos/ultimas/&lt;n&gt;</code> - Últimas N
mediciones (JSON)</li>
0198              <li><span class="badge">GET</span>  <code>/grafica</code> - Gráfica temporal (PNG)</li>
0199              <li><span class="badge">GET</span>  <code>/grafica/uma/&lt;id&gt;</code> - Gráfica de
una UMA (PNG)</li>
0200              <li><span class="badge">GET</span>  <code>/dashboard</code> - Dashboard web visual</li>
0201              <li><span class="badge">GET</span>  <code>/estadisticas</code> - Estadísticas por UMA (
JSON)</li>
0202          </ul>
0203      </div>
0204      <p><a href="/dashboard">Ver Dashboard</a> | <a href="/datos">Ver datos JSON</a></p>
0205  </body>
0206  </html>"""
0207
0208
0209  @app.route('/datos', methods=['POST'])
0210  def recibir_datos():
0211      try:
0212          data = request.get_json(force=True)
0213          if not data:
0214              return jsonify({'error': 'JSON inválido'}), 400
0215
0216          uma_id = int(data.get('id', 0))
0217          mp01 = int(data.get('mp01', 0))
0218          mp25 = int(data.get('mp25', 0))
0219          mp10 = int(data.get('mp10', 0))
0220          h03 = int(data.get('h03', 0))
0221          h05 = int(data.get('h05', 0))
0222          h10 = int(data.get('h10', 0))
0223          h25 = int(data.get('h25', 0))
0224          h50 = int(data.get('h50', 0))
0225          h100 = int(data.get('h100', 0))
0226          te = int(data.get('te', 0))
0227          hr = int(data.get('hr', 0))
0228          fecha = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
0229
0230          con = get_db()
0231          con.execute('''
0232              INSERT INTO mediciones
0233                (uma_id,mp01,mp25,mp10,h03,h05,h10,h25,h50,h100,te,hr,fecha)
0234              VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?)
0235          ''', (uma_id, mp01, mp25, mp10, h03, h05, h10, h25, h50, h100, te, hr, fecha))
0236          con.commit()
0237          con.close()
0238
0239          print(f"[{fecha}] UMA{uma_id} PM1={mp01} PM2.5={mp25} PM10={mp10} T={te}C HR={hr}%")
0240          return jsonify({'status': 'ok', 'fecha': fecha}), 201
0241      except Exception as e:
0242          return jsonify({'error': str(e)}), 500
0243
0244
0245  @app.route('/datos', methods=['GET'])
0246  def listar_datos():
0247      try:
0248          limit = request.args.get('limit', 100, type=int)
0249          con = get_db()
0250          rows = con.execute('SELECT * FROM mediciones ORDER BY id DESC LIMIT ?', (limit,)).fetchall(
)
0251          con.close()
0252          return jsonify([dict(r) for r in rows])
0253      except Exception as e:
0254          return jsonify({'error': str(e)}), 500
0255
0256
0257  @app.route('/datos/uma/<int:uma_id>', methods=['GET'])
0258  def datos_por_uma(uma_id):
0259      try:
0260          limit = request.args.get('limit', 100, type=int)
0261          con = get_db()
0262          rows = con.execute('SELECT * FROM mediciones WHERE uma_id=? ORDER BY id DESC LIMIT ?', (
uma_id, limit)).fetchall()
0263          con.close()
0264          return jsonify([dict(r) for r in rows])
0265      except Exception as e:
0266          return jsonify({'error': str(e)}), 500
```

```
0267
0268
0269  @app.route('/datos/ultimas/<int:n>', methods=['GET'])
0270  def ultimas_n(n):
0271      try:
0272          con = get_db()
0273          rows = con.execute('SELECT * FROM mediciones ORDER BY id DESC LIMIT ?', (n,)).fetchall()
0274          con.close()
0275          return jsonify([dict(r) for r in rows])
0276      except Exception as e:
0277          return jsonify({'error': str(e)}), 500
0278
0279
0280  @app.route('/estadisticas', methods=['GET'])
0281  def estadisticas():
0282      try:
0283          con = get_db()
0284          rows = con.execute('''
0285              SELECT
0286                  uma_id,
0287                  COUNT(*) as total,
0288                  ROUND(AVG(mp01),1) as avg_mp01,
0289                  ROUND(AVG(mp25),1) as avg_mp25,
0290                  ROUND(AVG(mp10),1) as avg_mp10,
0291                  ROUND(AVG(te),1)   as avg_te,
0292                  ROUND(AVG(hr),1)   as avg_hr,
0293                  MAX(fecha) as ultima_fecha
0294              FROM mediciones
0295              GROUP BY uma_id
0296              ORDER BY uma_id
0297          ''').fetchall()
0298          con.close()
0299          return jsonify([dict(r) for r in rows])
0300      except Exception as e:
0301          return jsonify({'error': str(e)}), 500
0302
0303
0304  @app.route('/grafica', methods=['GET'])
0305  def grafica_general():
0306      try:
0307          limit = request.args.get('limit', 50, type=int)
0308          con = get_db()
0309          rows = con.execute('SELECT * FROM mediciones ORDER BY id DESC LIMIT ?', (limit,)).fetchall(
)
0310          con.close()
0311          rows = list(reversed(rows))
0312          buf = generar_grafica_pm(rows, 'Todos los sensores UMA')
0313          return send_file(buf, mimetype='image/png', max_age=0)
0314      except Exception as e:
0315          return jsonify({'error': str(e)}), 500
0316
0317
0318  @app.route('/grafica/uma/<int:uma_id>', methods=['GET'])
0319  def grafica_uma(uma_id):
0320      try:
0321          limit = request.args.get('limit', 50, type=int)
0322          con = get_db()
0323          rows = con.execute('SELECT * FROM mediciones WHERE uma_id=? ORDER BY id DESC LIMIT ?', (
uma_id, limit)).fetchall()
0324          con.close()
0325          rows = list(reversed(rows))
0326          buf = generar_grafica_pm(rows, f'UMA{uma_id} — Material Particulado')
0327          return send_file(buf, mimetype='image/png', max_age=0)
0328      except Exception as e:
0329          return jsonify({'error': str(e)}), 500
0330
```

Explicación de los bloques principales

Línea / Bloque	Explicación
imports y app	Importa Flask, SQLite y Matplotlib; define la aplicación y la ruta datalogger.db.
init_db()	Crea la tabla mediciones si no existe. El campo id es autoincremental.
get_db()	Abre SQLite y activa Row para permitir dict(r) y acceso por nombre.
parse_fecha()	Convierte textos almacenados al formato datetime; usa la hora actual como respaldo.
generar_grafica_pm()	Dibuja tres paneles: PM, temperatura y humedad. Usa un bloqueo para evitar conflictos de Matplotlib.
generar_histograma()	Consulta promedios h03…h100 y construye barras por diámetro.
POST /datos	Obtiene JSON, convierte campos a enteros, agrega fecha y ejecuta INSERT parametrizado.
GET /datos	Devuelve las mediciones más recientes, limitado por el parámetro limit.
GET /datos/uma/<id>	Filtra por UMA con una consulta parametrizada.

Línea / Bloque	Explicación
GET /datos/ultimas/<n>	Devuelve las últimas n filas ordenadas por id descendente.
GET /estadisticas	Cuenta registros y calcula promedios por UMA con AVG y ROUND.
GET /grafica	Entrega PNG temporal; invierte las filas para dibujarlas en orden cronológico.
GET /grafica/uma/<id>	Entrega la gráfica solamente para una estación.
GET /grafica/histograma	Entrega el histograma, opcionalmente filtrado por UMA.
GET /dashboard	Consulta datos, codifica PNG en Base64 y genera el HTML con JavaScript.
main	Inicializa la base de datos y ejecuta Flask en 0.0.0.0:5000.

Línea crítica: creación de la base de datos

Línea / Bloque	Explicación
CREATE TABLE IF NOT EXISTS	La ejecución repetida del servidor no elimina ni duplica la tabla.
INTEGER PRIMARY KEY AUTOINCREMENT	Asigna una clave única y creciente para cada registro.
fecha TEXT NOT NULL	Fecha almacenada como texto en formato compatible con datetime.

Línea crítica: recepción y inserción JSON

Línea / Bloque	Explicación
request.get_json(force=True)	Fuerza el análisis del cuerpo como JSON aunque el cliente omita Content-Type.
data.get('id', 0)	Obtiene el valor del JSON; si falta, usa cero.
datetime.now().strftime(...)	Registra la fecha y hora al recibir la medición.
INSERT INTO mediciones VALUES (?,?,?,?,?)	Inserta los cinco valores en las posiciones protegidas por placeholders.
con.commit()	Confirma la escritura en disco.

8. Ejecución automatizada: archivos .bat

```
@echo off
cd /d "%~dp0serial_sim"
python server_flask.py
pause
```

iniciar_servidor_flask.bat cambia al directorio del proyecto usando la ruta del propio archivo, entra en serial_sim y ejecuta Flask. El pause mantiene abierta la ventana para revisar mensajes. iniciar_simulador.bat realiza lo mismo y lanza simu_trama.py COM1 0.1.

9. Ejecución completa paso a paso

Línea / Bloque	Explicación
1	Abrir proyecto2.vspe y activar el par de puertos virtuales COM1 ↔ COM2.
2	Ejecutar iniciar_servidor_flask.bat.
3	Comprobar http://127.0.0.1:5000 en un navegador.
4	Ejecutar iniciar_simulador.bat; COM1 debe enviar las tramas.
5	Abrir Lazarus_client.exe, seleccionar COM2 y pulsar Conectar.
6	Elegir UMA1…UMA5. Solo esa UMA alimentará las gráficas locales.
7	Activar Enviar a Flask para guardar cada trama en SQLite.
8	Abrir /dashboard para visualizar gráficos web, historial y estadísticas.

10. Conceptos clave para explicar en clase

Línea / Bloque	Explicación
Serialización	La trama convierte datos estructurados en texto separado por comas.
Frame de datos	El salto de línea '\n' permite saber dónde termina una medición.
Buffer serial	Acumula fragmentos recibidos hasta completar una trama.
Parser	Convierte texto CSV en variables numéricas.
Cliente-servidor	Lazarus actúa como cliente HTTP y Flask como servidor.
REST	Las rutas GET y POST representan operaciones sobre recursos de datos.
JSON	Formato textual usado para transportar una medición entre Lazarus y Flask.
SQLite	Base de datos local administrada por archivo, sin servidor externo.
Interfaces Lazarus	Controles visuales (.lfm) separados de la lógica Object Pascal (.pas).
Actualización no bloqueante	TTimer refresca gráficos cada 200 ms sin convertir el evento serial en trabajo gráfico pesado.

11. Resumen de rutas Flask

Línea / Bloque	Explicación
GET /	Página de ayuda con enlaces y rutas disponibles.
POST /datos	Recibe JSON y almacena una medición.
GET /datos	Lista mediciones; admite limit.
GET /datos/uma/<id>	Lista mediciones de una UMA.
GET /datos/ultimas/<n>	Lista las últimas n mediciones.
GET /estadisticas	Devuelve conteo y promedios por UMA.
GET /grafica	Devuelve un PNG con las últimas muestras.
GET /grafica/uma/<id>	Devuelve un PNG de una UMA.
GET /grafica/histograma	Devuelve un PNG de barras de partículas.
GET /dashboard	Devuelve el dashboard HTML.

Conclusión

El proyecto demuestra una cadena completa de adquisición, comunicación, almacenamiento y visualización. El componente Lazarus funciona como interfaz local de monitoreo, mientras Flask convierte los datos en un servicio accesible desde navegador y API. La explicación más importante es reconocer que la trama serial se delimita por salto de línea, se parsea en doce valores, se filtra por UMA y se empaqueta en JSON para persistirse en SQLite.